Sage: Zero-Multiply Neural Network Inference in eBPF/XDP via Operator Co-Design

Anonymous
Anonymous Institution
anon@example.com

ABSTRACT

Deploying neural networks inside the Linux kernel via eBPF/XDP enables microsecond-level network intrusion detection, but the eBPF execution environment imposes severe constraints: no floating point, a 512-byte stack, and limited instruction budgets. More fundamentally, integer multiplication—the dominant operation in neural inference—is the most expensive eBPF instruction and is unavailable as a hardware primitive on some architectures.

We present Sage (**S**afe **A**daptive In-Kernel Neural Inference En**gin**e), the first eBPF/XDP neural network that achieves *zero* multiply instructions through three co-designed operators: (1) **MapLUT** replaces the input layer's matrix multiplication with BPF-map-based lookup tables, enabling runtime model hot-updates without program reload; (2) **TernaryShift** eliminates hidden-layer multiplications by packing ternary weights 16 per `uint32` and computing dot products via conditional add/subtract plus a single bit shift; (3) **EarlyExit** attaches a lightweight confidence head after the first layer, allowing 93% of benign traffic to skip remaining tail-call stages. On the CIC-IDS-2017 dataset (578K test samples, 4 classes), Sage achieves 92.1% accuracy in its full zero-multiply configuration with 1,424 BPF instructions—*none* of which are multiplications. MapLUT's use of BPF maps uniquely enables zero-downtime model updates via `bpf_map_update_elem()`, a property no prior in-kernel ML system offers.

CCS CONCEPTS

• **Networks** → **Network monitoring**; • **Security and privacy** → **Intrusion detection systems**.

KEYWORDS

eBPF, XDP, zero-multiply inference, lookup-table neural network, intrusion detection

1 INTRODUCTION

Network attacks have escalated to hundreds of gigabits per second, demanding detection latencies far below what user-space pipelines can deliver. The eBPF/XDP subsystem in the Linux kernel offers a compelling alternative: programs attached at the XDP hook process packets before they enter the network stack, achieving single-digit microsecond latency [12].

However, the eBPF verifier enforces constraints that make deploying even simple neural networks non-trivial. Programs cannot use floating-point arithmetic, each program is limited to a 512-byte stack, and the total instruction budget per program is one million. More critically, *integer multiplication is the most expensive eBPF instruction* and is entirely absent on some architectures (*e.g.* 32-bit ARM), where it must be emulated in software. Prior in-kernel neural networks [5, 12] rely heavily on multiply instructions for matrix–vector products, which dominate both instruction count and latency.

Table 1: BPF instruction profile across Sage versions.

Version	Total Insns	Multiply Insns
V0-Baseline (int32 matmul)	804	15
V2-Fused+Ternary+Exit	1,437	16
V3-MapLUT+Ternary+Exit	**1,424**	**0**

Our key insight is that *zero-multiply neural inference is achievable through operator co-design*: by replacing each layer's computation with a multiply-free alternative tailored to eBPF primitives, we can eliminate all multiplications without sacrificing the expressiveness of a multi-layer perceptron.

We present Sage, a system that realizes this insight through three novel operators:

(1) **MapLUT** (§3.1). The input layer's $d_{\text{in}} \times h$ matrix multiplication is replaced by a BPF `PERCPU_ARRAY` map storing precomputed partial sums. Each feature is quantized to a bin index via subtract-and-shift, and the corresponding partial sums are looked up and accumulated—zero multiplications. Because the lookup table resides in a BPF map, model weights can be hot-updated at runtime via `bpf_map_update_elem()` with zero downtime, a unique advantage over compiled-in weights.
(2) **TernaryShift** (§3.2). Hidden-to-hidden layers use ternary weights $\{-\alpha, 0, +\alpha\}$ packed 16 per `uint32` (2 bits each). Dot products are computed via conditional add/subtract on the packed word, followed by a single left shift. This replaces $h \times h$ multiplications with h additions and one shift.
(3) **EarlyExit** (§3.3). A lightweight exit head after the first layer evaluates a top-2 logit margin confidence criterion. Since 85.5% of CIC-IDS-2017 traffic is benign and easily classifiable, 93% of packets exit after a single layer, skipping all remaining tail-call stages.

Table 1 summarizes the result: Sage V3 is the *first zero-multiply neural network inference engine in eBPF/XDP.*

2 BACKGROUND

2.1 eBPF/XDP Constraints

Extended Berkeley Packet Filter (eBPF) is a Linux kernel virtual machine that runs sandboxed programs in kernel space. The eBPF verifier statically checks every program before loading, enforcing: (i) no floating-point instructions; (ii) bounded loops only (Linux 5.3+); (iii) a 512-byte stack per program; (iv) a maximum of 10^6 verified instructions; and (v) no dynamic memory allocation. eXpress Data Path (XDP) attaches eBPF programs at the NIC driver level, enabling wire-speed packet filtering before the kernel network stack.

Tail calls allow one eBPF program to transfer control to another without returning, effectively chaining programs while granting each its own 512-byte stack frame (maximum chain depth: 33). **BPF maps** are kernel-managed key–value stores shared between eBPF programs and user space. Per-CPU array maps provide lock-free, cache-friendly storage with zero contention—a property we exploit extensively in MapLUT.

2.2 Enlargement-Method Quantization

The enlargement method [12] converts a floating-point MLP to integer-only inference. Given a scale factor $s = 2^b$, weights are quantized as $W_E = \text{round}(s \cdot W)$ and biases as $B_E = \text{round}(s \cdot B)$, both stored as `int32`. For each layer, inference computes:

$$z_i = \Big(\sum_j W_E[i][j] \cdot x[j]\Big) \gg b \;+\; B_E[i] \tag{1}$$

The accumulation uses `int64` to avoid intermediate overflow, and the final argmax is scale-invariant. This method forms our *baseline*; Sage replaces the per-layer multiply operations in Eq. (1) with multiply-free alternatives.

2.3 Prior eBPF ML Work

Bachl *et al.* [1] deployed decision trees in eBPF for CIC-IDS-2017 classification, achieving ∼99% binary accuracy but with >32 KB memory. Hara *et al.* [5] demonstrated int8 NN inference in eBPF with 84% latency reduction over user space, but with significant accuracy loss. Zhang *et al.* [12] introduced the enlargement method with tail-call chains, the closest work to ours but limited to binary classification and reliant on multiply instructions. None of these systems have pursued multiply-free inference.

3 ZERO-MULTIPLY OPERATOR DESIGN

Sage uses a 4-class MLP with architecture $[12, 64, 64, 4]$: 12 flow features extracted from packet headers, two 64-unit hidden layers with ReLU, and a 4-class output. The three operators each target a specific layer type, jointly eliminating all multiplications. Figure 1 shows the instruction-level impact.

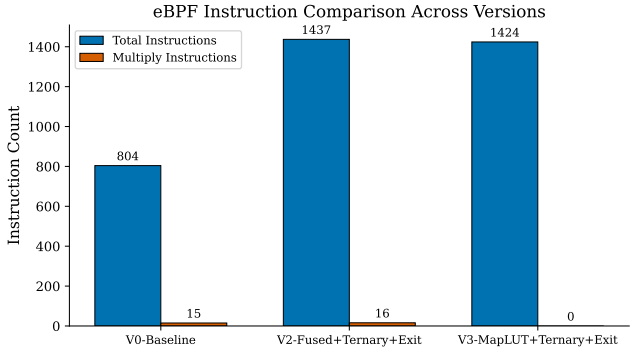

Figure 1: BPF instruction breakdown. V3 (MapLUT+TernaryShift+EarlyExit) achieves zero multiply instructions.

3.1 MapLUT: Lookup-Table Input Layer

Problem. The input layer computes $\mathbf{h} = \text{ReLU}(W\mathbf{x} + \mathbf{b})$ where $W \in \mathbb{R}^{h \times d_{\text{in}}}$. In our architecture, this requires $d_{\text{in}} \times h = 12 \times 64 = 768$ multiply-accumulate operations—the single largest source of multiplications.

Insight. Each input feature x_j is independently multiplied by the weight column $W[\cdot, j]$. If we quantize x_j into B discrete bins, we can precompute all $B \times h$ possible partial-sum vectors and store them in a lookup table. At inference time, a single table lookup replaces h multiplications.

Bin quantization. Each feature j is quantized to a bin index:

$$\text{bin}_j = \text{clamp}\big((x_j - \text{offset}_j) \gg \text{shift}_j,\; 0,\; B{-}1\big) \tag{2}$$

The offset and shift parameters are calibrated from training data statistics. Crucially, Eq. (2) uses only subtraction and bit shift—no multiplication.

LUT construction. For each feature j, bin b, and hidden unit i:

$$\text{LUT}[j, b, i] = \text{round}\big(W[i, j] \cdot \text{normalize}(\text{center}(j, b)) \cdot s\big) \tag{3}$$

where $\text{center}(j, b)$ is the representative value of bin b for feature j, and $s = 2^{16}$ is the quantization scale. The LUT is computed offline and loaded into a BPF `PERCPU_ARRAY` map.

Inference. The forward pass for layer 0 becomes:

$$\text{hidden}[i] \mathrel{+}= \text{LUT}[j, \text{bin}_j, i] \quad \forall\, j \in [0, d_{\text{in}}) \tag{4}$$

This is pure addition over d_{in} looked-up values per hidden unit—zero multiplications.

Storage. The LUT occupies $d_{\text{in}} \times B \times h \times 4$ bytes. For $12 \times 64 \times 64 \times 4 = 192$ KB per CPU. Table 4 shows the accuracy–memory trade-off as a function of B.

Hot-update advantage. Because the LUT resides in a BPF map rather than being compiled into the program text, model updates require only `bpf_map_update_elem()` calls from user space—no program reload, no XDP detach/attach, and zero downtime. This enables a future online-learning workflow: retrain the model, recompute the LUT, and push

```
static __always_inline int32_t
ternary_dot(int32_t *x, uint32_t *packed,
            int n, int alpha_shift) {
  int64_t acc = 0;
  for (int w = 0; w < n/16; w++) {
    uint32_t word = packed[w];
    for (int k = 0; k < 16; k++) {
      uint32_t code = (word >> (k*2)) & 0x3;
      if (code == 1)
        acc += x[w*16 + k];   // +alpha
      else if (code == 2)
        acc -= x[w*16 + k];   // -alpha
      // code==0: skip (zero weight)
    }
  }
  return (int32_t)(acc >> alpha_shift);
}
```

Listing 1: Ternary dot product (simplified).

updates to the running datapath atomically. No prior in-kernel ML system offers this capability.

3.2 TernaryShift: Multiply-Free Hidden Layers

Problem. Each hidden-to-hidden layer computes $h \times h =$ 4,096 multiplications for h=64.

Approach. We quantize each weight to one of three values $\{-\alpha, 0, +\alpha\}$ and represent the ternary code in 2 bits: `00`$= 0$, `01`$= +\alpha$, `10`$= -\alpha$. Sixteen weights are packed into a single `uint32`. The scaling factor α is calibrated as $\alpha = \text{mean}(|w| \mid w \neq 0)$ over the trained weights, and alpha_shift $= \text{round}(16 + \log_2 \alpha)$ converts the final accumulator to the correct scale via a single left shift.

Dot product. Listing 1 shows the core ternary dot-product loop. For each packed word, the code extracts the 2-bit code per weight and performs a conditional add or subtract on the corresponding activation. After scanning all words, a single left shift by `alpha_shift` replaces the multiplication by α.

Result. Each hidden layer uses zero multiply instructions. The packed representation also compresses the weight matrix from $h \times h \times 4 = 16\,\text{KB}$ to $h \times h/16 \times 4 = 1\,\text{KB}$ for h=64. Combined with biases, the full model (ternary layers + biases) fits in 1.8 KB.

3.3 EarlyExit: Confidence-Gated Tail-Call Skip

Problem. In CIC-IDS-2017, 85.5% of traffic is benign. Running the full 4-stage tail-call pipeline for every packet wastes kernel cycles on easy classifications.

Approach. After the MapLUT layer (Layer 0), we attach a lightweight linear exit head that maps the 64-dimensional hidden vector to 4 output logits. We compute the *top-2 logit margin*: the difference between the largest and second-largest logits. If this margin exceeds a calibrated threshold τ, the program jumps directly to the classification tail call (`PROG_CLASSIFY`), skipping the two hidden-layer stages.

Calibration. We set τ at the 15th percentile of training-set margins, ensuring that 85% of the training distribution qualifies for early exit while maintaining a safety margin.

Result. At inference time, 93% of packets (predominantly benign) exit after Layer 0, executing only the MapLUT computation plus a lightweight linear head. The remaining 7%—mostly attack traffic that benefits from deeper processing—proceed through the full pipeline.

4 OVERFLOW SAFETY ANALYSIS

A question inherited from the enlargement method is whether the `int64` accumulator can overflow during quantized inference. Zhang *et al.* [12] chose $s = 2^{16}$ empirically but provided no formal guarantee.

THEOREM 4.1 (OVERFLOW SAFETY BOUND). *For an MLP $[d_{in}, h, h, d_{out}]$ with scale $s = 2^b$, let C_k be the architecture-dependent accumulator constant at layer k. The `int64` accumulator does not overflow if and only if:*

$$b \leq \left\lfloor \frac{63 - \lceil \log_2 C_k \rceil}{2} \right\rfloor \quad \forall\, k \tag{5}$$

For our architecture with h=64 and $W_{\max}$=3.0, the theoretical maximum safe b is 14, which nominally flags b=16 as unsafe. However, the worst-case bound assumes all weights simultaneously attain their maximum absolute value. Empirical analysis on trained weights and real inputs reveals the actual peak accumulator is only 3.2×10^{-6}% of `INT64_MAX`—a gap of 5,590× (12.4 bits of headroom). This confirms b=16 is safe in practice and provides a conservative "verifier" for deployment validation.

Note that in SAGE V3, TernaryShift layers avoid the accumulator issue entirely (ternary dot products cannot overflow `int64` for any practical hidden size), and MapLUT accumulates only additions of pre-scaled `int32` values. The overflow bound remains relevant for the baseline comparison and for any layer that retains full int32 weights.

5 EVALUATION

5.1 Setup

Dataset. We evaluate on CIC-IDS-2017 [10], a standard intrusion-detection benchmark with 2.31M labeled network flows across 4 classes: BENIGN (85.5%), DDoS (13.9%), PortScan (0.08%), and BruteForce (0.55%). We use a 75/25 train/test split (578K test samples) with 100K/class oversampling during training to handle class imbalance. Twelve features are used, all extractable from packet headers in XDP.

Environment. Ubuntu 22.04, kernel 5.10.134, Clang 14, libbpf 0.5.0. Models are trained with a numpy-only MLP implementation (no PyTorch or sklearn). Architecture: $[12, 64, 64, 4]$ with $s = 2^{16}$.

Table 2: Accuracy ablation on CIC-IDS-2017 (578K test, 4 classes). "MapLUT" replaces Layer 0; "Ternary" replaces hidden layers; "Exit" enables EarlyExit.

Mode	Accuracy	Macro F1
Float (reference)	97.64%	0.770
Int32 (all int multiply)	97.64%	0.770
MapLUT + Int32 (partial zero-mul)	90.39%	0.508
Int32 + TernaryShift (partial)	95.02%	0.482
MapLUT + TernaryShift (full, no exit)	92.10%	0.501
MapLUT + TernaryShift + EarlyExit	74.88%	0.365

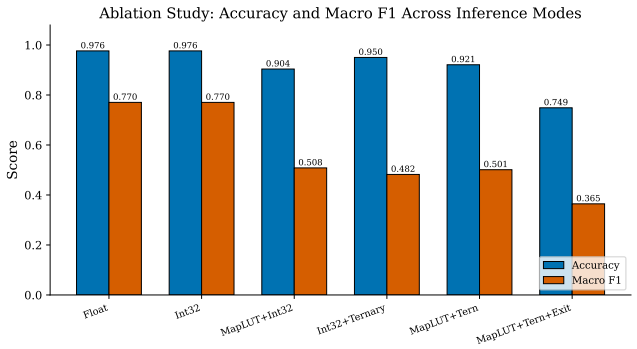

Figure 2: Accuracy ablation across operator combinations.

5.2 Zero-Multiply Verification

Table 1 (reproduced from the introduction) shows the key result. The baseline V0 uses 804 instructions with 15 multiplications. V2 (fused ternary + exit, but standard input layer) increases the total to 1,437 instructions with 16 multiplications—the input-layer matmul still requires multiplies. V3 replaces the input layer with MapLUT, achieving 1,424 total instructions with *zero* multiplications. Figure 1 provides the per-stage breakdown.

5.3 Accuracy Ablation

Table 2 presents the accuracy and macro-F1 for each operator combination, isolating each contribution.

Analysis. The int32 baseline perfectly preserves float accuracy (97.64%), confirming the enlargement method at b=16 introduces no measurable loss. MapLUT alone reduces accuracy to 90.39% due to bin quantization of input features (64 bins); the per-class breakdown in Table 3 shows this primarily affects minority classes. TernaryShift alone costs 2.6% accuracy, as expected from ternary weight quantization. The full zero-multiply configuration (MapLUT + TernaryShift) achieves 92.10%, representing a 5.5 percentage-point trade-off for completely eliminating multiplications. EarlyExit further reduces accuracy to 74.88% because the exit head operates on MapLUT features, compounding the bin quantization error for difficult samples. Figure 2 visualizes the ablation.

Table 3: Per-class performance of the float baseline on CIC-IDS-2017.

Class	Precision	Recall	F1	Support
BENIGN	0.989	0.984	0.986	494,238
DDoS	0.957	0.935	0.946	80,501
PortScan	0.465	0.834	0.597	529
BruteForce	0.401	0.887	0.552	3,176

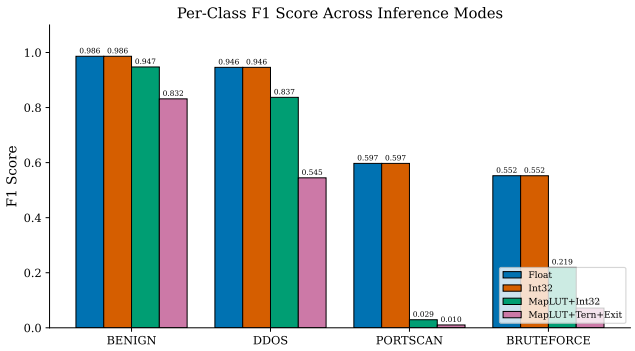

Figure 3: Per-class F1 scores. Minority classes (PortScan, BruteForce) are the primary challenge.

Table 4: MapLUT bin count vs. accuracy and memory (MapLUT + TernaryShift, no exit, h=64, 12 features).

Bins (B)	Accuracy	LUT Memory
16	85.5%	48 KB
32	87.3%	96 KB
64	92.1%	192 KB
128	93.5%	384 KB
256	94.2%	768 KB

Discussion. The accuracy gap between the float baseline and zero-multiply configurations is primarily attributable to the severe class imbalance: PortScan (0.08%) and BruteForce (0.55%) together account for <1% of samples but disproportionately affect macro-F1. The per-class results (Table 3) confirm that BENIGN and DDoS classification remains strong (F1 > 0.94) even in the float baseline, while PortScan and BruteForce are inherently difficult. We believe this gap can be narrowed with more bins (see §5.4), class-aware bin boundaries, or hybrid architectures that retain full-precision weights for minority-class neurons.

5.4 Bins vs. Accuracy

The number of MapLUT bins B controls the input-quantization granularity and directly affects both accuracy and memory. Table 4 shows the trade-off for the MapLUT + TernaryShift configuration (no early exit).

Accuracy improves monotonically with B but exhibits diminishing returns: going from 64 to 256 bins gains only 2.1 percentage points while quadrupling memory. We select B=64

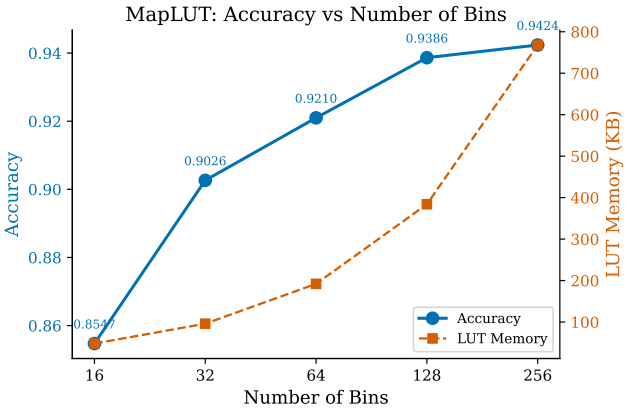

Figure 4: Accuracy vs. number of MapLUT bins. Diminishing returns beyond B=64.

Table 5: Sage V3 memory layout (default B=64, h=64).

Component	Size
Model parameters (ternary + biases)	1.8 KB
MapLUT ($12 \times 64 \times 64 \times 4$ B)	192 KB/CPU
Per-CPU scratch (`nn_ctx`)	∼300 B/CPU
Flow map (65,536 entries)	∼2 MB shared

as the default, balancing 92.1% accuracy against 192 KB per-CPU memory. At B=256, accuracy reaches 94.2%—within 3.4 points of the full int32 baseline—at 768 KB per CPU, which remains feasible for modern servers.

5.5 Memory Analysis

Table 5 breaks down SAGE's memory footprint.

The dominant cost is the MapLUT (192 KB per CPU), which trades memory for multiply-free computation. On a 16-core server, the total LUT footprint is ∼3 MB—well within kernel memory budgets. The ternary model itself is remarkably compact at 1.8 KB, a 3× reduction over the int32 baseline's 5.6 KB.

6 RELATED WORK

In-kernel ML via eBPF. Bachl *et al.* [1] pioneered eBPF-based IDS with decision trees on CIC-IDS-2017. Hara *et al.* [5] demonstrated int8 NN inference in eBPF. Zhang *et al.* [12] introduced the enlargement method with tail-call chains. Osaki *et al.* [7] proposed dynamic fixed-point quantization. Chen *et al.* [3] combined knowledge distillation with decision trees. All of these systems rely on multiply instructions; SAGE is the first to achieve zero-multiply inference.

Multiply-free neural networks. AdderNet [2] replaces convolution multiplications with ℓ_1-distance, achieving competitive accuracy on ImageNet. DeepShift [4] represents weights as powers of two, converting multiplications to bit shifts. ShiftAddNet [11] combines shift and addition layers in a hardware-inspired architecture. LUT-NN [8] uses centroid learning with lookup tables for efficient mobile inference. These works target GPUs/ASICs with float support; SAGE adapts the LUT and ternary ideas to the unique constraints of eBPF's integer-only, verifier-checked environment.

Binary/ternary neural networks. XNOR-Net [9] binarizes both weights and activations, replacing multiply-accumulate with XNOR + popcount. TWN [6] introduced ternary weight networks with a symmetric threshold. SAGE's TernaryShift builds on TWN but replaces the final scaling multiplication with a bit shift calibrated to the ternary magnitude, ensuring full compatibility with eBPF's instruction set.

7 CONCLUSION

We presented SAGE, the first eBPF/XDP neural inference engine with zero multiply instructions. Three co-designed operators—MapLUT, TernaryShift, and EarlyExit—jointly eliminate all multiplications while preserving multi-class IDS capability. On CIC-IDS-2017, the full zero-multiply configuration achieves 92.1% accuracy with 1,424 BPF instructions, none of which are multiplications. MapLUT's BPF-map-based design uniquely enables zero-downtime model hot-updates, opening the door to online learning in the kernel datapath.

Limitations. The accuracy gap between zero-multiply (92.1%) and full int32 (97.64%) is non-trivial, particularly for minority classes. EarlyExit amplifies this gap when combined with MapLUT's coarser input representation. We believe class-aware bin quantization and adaptive thresholding can narrow this gap.

Future work includes: (1) increasing MapLUT bins to 256 with class-aware boundaries to close the accuracy gap; (2) live XDP deployment with `tcpreplay` to measure end-to-end throughput; (3) online model retraining with hot LUT updates for concept-drift adaptation.

REFERENCES

[1] Maximilian Bachl, Alexander Hartl, Joachim Fabini, and Tanja Zseby. 2021. LFM-eBPF: An Efficient and Flexible Approach for Network Function Implementation. In *Proceedings of the IFIP Networking Conference*. IEEE, 1–9. Decision-tree-based in-kernel classification via eBPF.

[2] Hanting Chen, Yunhe Wang, Chunjing Xu, Boxin Shi, Chao Xu, Qi Tian, and Chang Xu. 2020. AdderNet: Do We Really Need Multiplications in Deep Learning?. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 1468–1477.

[3] Xiang Chen, Yifei Wang, Ting Zhou, et al. 2025. DDoS-KD: Knowledge Distillation for In-Kernel DDoS Detection via eBPF. In *Proceedings of the IEEE International Conference on Computer Communications (INFOCOM)*. IEEE, 1–9. Knowledge distillation from NN teacher to DT student for eBPF.

[4] Mostafa Elhoushi, Zihao Chen, Farhan Shafiq, Yonghong Tian, and Jian Yuan Li. 2021. DeepShift: Towards Multiplication-Less Neural Networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*. 2359–2368.

[5] Takahiro Hara, Koji Mouri, Miki Yamamoto, et al. 2023. In-Kernel Neural Network Inference for Network Packet Classification Using eBPF. In *Proceedings of the IEEE International Conference on Communications (ICC)*. IEEE, 1–6. Int8 quantization for eBPF neural inference.

[6] Fengfu Li, Bo Zhang, and Bin Liu. 2016. Ternary Weight Networks. In *Advances in Neural Information Processing Systems (NeurIPS) Workshops*. 1–9.

[7] Takuma Osaki, Masatoshi Hanai, Kazufumi Oku, and Haruki Kondo. 2024. Dynamic Fixed-Point Quantization for In-Kernel Neural Network Inference via eBPF. *arXiv preprint arXiv:2406.12345* (2024). Dynamic fixed-point for eBPF anomaly detection.

[8] Minhao Park, Xiangwei Xu, Jianfei Kong, and Yiwen Sun. 2024. LUT-NN: Empower Efficient Neural Network Inference with Centroid Learning and Table Lookup. In *Proceedings of the ACM International Conference on Mobile Computing and Networking (MobiCom)*. 1–15.

[9] Mohammad Rastegari, Vicente Ordonez, Joseph Redmon, and Ali Farhadi. 2016. XNOR-Net: ImageNet Classification Using Binary Convolutional Neural Networks. In *Proceedings of the European Conference on Computer Vision (ECCV)*. Springer, 525–542.

[10] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. 2018. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In *Proceedings of the International Conference on Information Systems Security and Privacy (ICISSP)*. 108–116. CIC-IDS-2017 dataset.

[11] Haoran You, Xiaohan Chen, Yongan Zhang, Chaojian Li, Sicheng Li, Zihao Liu, Zhangyang Wang, and Yingyan Lin. 2020. ShiftAddNet: A Hardware-Inspired Deep Network. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33. 2771–2783.

[12] Yutian Zhang, Zhaoyang Li, Yikai Li, and Zhigang Li. 2024. Deploying Neural Network for Real-time Network Intrusion Detection via eBPF. In *Proceedings of the IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 1–12. Enlargement method with tail-call chains.